

SOC Alert Automation Project Report

Wazuh + n8n + AbuseIPDB + Telegram

1. Project Overview

This project demonstrates a practical SOC alert automation pipeline using Wazuh, n8n, AbuseIPDB, and Telegram.

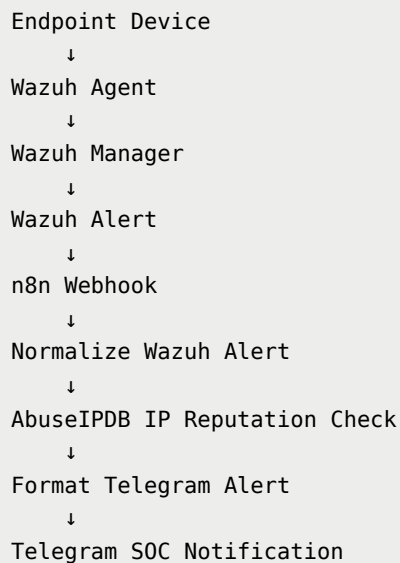
The main goal of the project was to detect suspicious activity from monitored endpoints, normalize the alert data, enrich suspicious IP addresses using threat intelligence, and forward the final alert to Telegram in a clean SOC-style format.

The project was implemented in a lab environment using:

Component	Purpose
Ubuntu Server	Main SOC server
Wazuh Manager	Central security monitoring and log analysis
Wazuh Indexer	Alert storage and indexing
Wazuh Dashboard	Web interface for alert investigation
Wazuh Agents	Endpoint log collection
Windows Endpoint	Windows failed login detection
Kali Linux Endpoint	SSH failed login / attack simulation
n8n	Automation and alert workflow processing
AbuseIPDB	IP reputation and threat intelligence enrichment
Telegram Bot	Real-time SOC alert notification

The project was designed to be scalable. It is not limited to one device only. Any endpoint where the Wazuh Agent is installed and enrolled into the Wazuh Manager can become part of the monitored environment.

2. Architecture



3. Agent-Based Monitoring Concept

The project uses an agent-based monitoring model.

This means that each endpoint sends security events to the central Wazuh Manager through a Wazuh Agent. Once the agent is installed and registered, the endpoint becomes part of the monitoring scope.

In this lab, the monitored endpoints included:

Device	Role	Example Detection
Windows Machine	Endpoint monitoring	Failed login event
Kali Linux Machine	Attack simulation / Linux endpoint	SSH failed login
Ubuntu Server	SOC infrastructure	Wazuh, n8n, and services

The important point is that the monitoring logic is reusable.

If a new machine is added later, the same process applies:

1. Install Wazuh Agent.
2. Configure the Wazuh Manager IP.

3. Enroll the agent.
4. Restart the agent service.
5. Confirm the agent is active.
6. Generate a test security event.
7. Validate that the event appears in Wazuh.
8. Confirm that n8n processes and forwards the alert.

4. n8n Workflow Overview

The n8n workflow was responsible for receiving, processing, enriching, and forwarding alerts.

Workflow nodes:

```
Webhook
→ Normalize Wazuh Alert
→ AbuseIPDB Check
→ Format Telegram Alert
→ HTTP Request / Telegram Send Message
```

Node Purpose

n8n Node	Description
Webhook	Receives alert payload from Wazuh or test input
Normalize Wazuh Alert	Extracts useful fields from raw alert data
AbuseIPDB Check	Checks source IP reputation
Format Telegram Alert	Builds a readable SOC alert message
HTTP Request	Sends the alert to Telegram

5. n8n JavaScript Code

5.1 Normalize Wazuh Alert Node

Node name: Normalize Wazuh Alert

Node type: Code Node / Function Node

Purpose: Extract important fields from the raw Wazuh alert and prepare clean data for the next nodes.

```
// Normalize Wazuh Alert
// This code extracts the important fields from a Wazuh alert
// and prepares a clean object for enrichment and Telegram fo
rmatting.

const input = $json;

// Wazuh alerts may arrive directly or inside body depending
on webhook structure
const alert = input.body || input;

// Extract agent information
const agentName = alert.agent?.name || "Unknown Host";
const agentId = alert.agent?.id || "Unknown Agent ID";
const agentIp = alert.agent?.ip || "Unknown Agent IP";

// Extract rule information
const ruleId = alert.rule?.id || "N/A";
const ruleLevel = alert.rule?.level || "N/A";
const ruleDescription = alert.rule?.description || "No descri
ption available";

// Extract timestamp
const timestamp = alert.timestamp || new Date().toISOString
();

// Extract source IP from different possible Wazuh fields
const sourceIp =
  alert.data?.srcip ||
  alert.data?.src_ip ||
```

```

    alert.data?.win?.eventdata?.ipAddress ||
    alert.data?.srcaddr ||
    "N/A";

// Extract username if available
const username =
    alert.data?.dstuser ||
    alert.data?.srcuser ||
    alert.data?.win?.eventdata?.targetUserName ||
    alert.data?.win?.eventdata?.subjectUserName ||
    "N/A";

// Extract event type
const eventType =
    alert.data?.win?.system?.eventID ||
    alert.rule?.groups?.join(", ") ||
    "N/A";

// Extract MITRE information
const mitreIds = alert.rule?.mitre?.id || [];
const mitreTactics = alert.rule?.mitre?.tactic || [];
const mitreTechniques = alert.rule?.mitre?.technique || [];

// Build normalized alert object
return [
    {
        json: {
            timestamp,
            agentName,
            agentId,
            agentIp,
            sourceIp,
            username,
            eventType,
            ruleId,
            ruleLevel,

```

```

        ruleDescription,
        mitreIds: Array.isArray(mitreIds) ? mitreIds.join(", ")
: mitreIds,
        mitreTactics: Array.isArray(mitreTactics) ? mitreTactics.join(", ") : mitreTactics,
        mitreTechniques: Array.isArray(mitreTechniques) ? mitreTechniques.join(", ") : mitreTechniques,
        rawAlert: alert
    }
}
];

```

Explanation

This code handles different Wazuh alert formats.

For example, Windows alerts and Linux SSH alerts may store the source IP and username in different fields. The code checks multiple possible locations to avoid losing important information.

It extracts:

```

Host name
Agent ID
Agent IP
Source IP
Username
Rule ID
Rule level
Rule description
MITRE mapping
Timestamp
Raw alert

```

5.2 AbuseIPDB Request Preparation Code

Node name: Prepare AbuseIPDB Check

Purpose: Prepare the IP address before sending it to AbuseIPDB.

```
// Prepare AbuseIPDB Check
// This code validates the source IP before sending it to AbuseIPDB.
```

```
const alert = $json;
const sourceIp = alert.sourceIp;
```

```
// Basic validation
const isValidIp =
  sourceIp &&
  sourceIp !== "N/A" &&
  sourceIp !== "127.0.0.1" &&
  sourceIp !== "::1" &&
  !sourceIp.startsWith("192.168.") &&
  !sourceIp.startsWith("10.") &&
  !sourceIp.startsWith("172.16.") &&
  !sourceIp.startsWith("172.17.") &&
  !sourceIp.startsWith("172.18.") &&
  !sourceIp.startsWith("172.19.") &&
  !sourceIp.startsWith("172.20.") &&
  !sourceIp.startsWith("172.21.") &&
  !sourceIp.startsWith("172.22.") &&
  !sourceIp.startsWith("172.23.") &&
  !sourceIp.startsWith("172.24.") &&
  !sourceIp.startsWith("172.25.") &&
  !sourceIp.startsWith("172.26.") &&
  !sourceIp.startsWith("172.27.") &&
  !sourceIp.startsWith("172.28.") &&
  !sourceIp.startsWith("172.29.") &&
  !sourceIp.startsWith("172.30.") &&
  !sourceIp.startsWith("172.31.");
```

```
return [
  {
    json: {
```

```

    ...alert,
    isPublicIp: isValidIp,
    abuseIpdbUrl: isValidIp
      ? `https://api.abuseipdb.com/api/v2/check?ipAddress=
${sourceIp}&maxAgeInDays=90`
      : null
  }
}
];

```

Explanation

This code prevents private/internal IP addresses from being sent to AbuseIPDB.

Private IPs like:

```

192.168.x.x
10.x.x.x
172.16.x.x - 172.31.x.x
127.0.0.1

```

are internal network addresses and usually should not be checked as public threat intelligence indicators.

5.3 AbuseIPDB Result Normalization Code

Node name: `Normalize AbuseIPDB Result`

Purpose: Extract risk score and reputation data from AbuseIPDB response.

```

// Normalize AbuseIPDB Result
// This code extracts the useful fields from AbuseIPDB response
// and merges them with the original normalized Wazuh alert.

const alert = $items("Normalize Wazuh Alert")[0].json;
const abuseResponse = $json;

```



```

let abuseConfidenceScore = "N/A";
let totalReports = "N/A";
let countryCode = "N/A";
let isp = "N/A";
let domain = "N/A";
let usageType = "N/A";

if (abuseResponse?.data) {
  abuseConfidenceScore = abuseResponse.data.abuseConfidenceScore ?? "N/A";
  totalReports = abuseResponse.data.totalReports ?? "N/A";
  countryCode = abuseResponse.data.countryCode ?? "N/A";
  isp = abuseResponse.data.isp ?? "N/A";
  domain = abuseResponse.data.domain ?? "N/A";
  usageType = abuseResponse.data.usageType ?? "N/A";
}

let riskLevel = "Unknown";

if (abuseConfidenceScore === "N/A") {
  riskLevel = "Not Checked";
} else if (abuseConfidenceScore >= 75) {
  riskLevel = "High";
} else if (abuseConfidenceScore >= 40) {
  riskLevel = "Medium";
} else if (abuseConfidenceScore >= 10) {
  riskLevel = "Low";
} else {
  riskLevel = "Clean";
}

return [
  {
    json: {
      ...alert,

```

```

        abuseConfidenceScore,
        totalReports,
        countryCode,
        isp,
        domain,
        usageType,
        riskLevel
    }
}
];

```

Explanation

This code converts the AbuseIPDB response into simple SOC-friendly fields:

```

Abuse Confidence Score
Total Reports
Country
ISP
Domain
Usage Type
Risk Level

```

Risk classification:

AbuseIPDB Score	Risk Level
75–100	High
40–74	Medium
10–39	Low
0–9	Clean
N/A	Not Checked

5.4 Format Telegram Alert Code

Node name: `Format Telegram Alert`

Purpose: Convert the normalized alert into a readable Telegram message.

```
// Format Telegram Alert
// This code creates a clean SOC-style Telegram alert message.
```

```
const alert = $json;
```

```
const riskEmoji = {
  High: "🔴",
  Medium: "🟠",
  Low: "🟡",
  Clean: "🟢",
  "Not Checked": "⚪",
  Unknown: "⚪"
};
```

```
const severityEmoji =
  alert.ruleLevel >= 12 ? "🚨" :
  alert.ruleLevel >= 8 ? "⚠️" :
  alert.ruleLevel >= 4 ? "🔍" :
  "ℹ️";
```

```
const emoji = riskEmoji[alert.riskLevel] || "⚪";
```

```
const message = `
${severityEmoji} *SOC Security Alert*
```

```
*Detection Summary*
```

- Host: \${alert.agentName}
- Agent ID: \${alert.agentId}
- Agent IP: \${alert.agentIp}
- Timestamp: \${alert.timestamp}

```
*Alert Details*
```

- Rule ID: \${alert.ruleId}
- Rule Level: \${alert.ruleLevel}

```

• Description: ${alert.ruleDescription}
• Event Type: ${alert.eventType}
• Username: ${alert.username}

*Network Indicator*
• Source IP: ${alert.sourceIp}

*Threat Intelligence - AbuseIPDB*
${emoji} Risk Level: ${alert.riskLevel}
• Abuse Confidence Score: ${alert.abuseConfidenceScore}
• Total Reports: ${alert.totalReports}
• Country: ${alert.countryCode}
• ISP: ${alert.isp}
• Domain: ${alert.domain}
• Usage Type: ${alert.usageType}

*MITRE ATT&CK Mapping*
• Technique ID: ${alert.mitreIds || "N/A"}
• Tactic: ${alert.mitreTactics || "N/A"}
• Technique: ${alert.mitreTechniques || "N/A"}

*Recommended Analyst Action*
1. Validate whether the login attempt was legitimate.
2. Check if the source IP appears in other alerts.
3. Review the affected endpoint logs.
4. Block the IP if confirmed malicious.
5. Escalate the incident if repeated attempts are detected.
`;

return [
  {
    json: {
      chat_id: "YOUR_TELEGRAM_CHAT_ID",
      text: message,
      parse_mode: "Markdown"
    }
  }
]

```

```
}  
];
```

Explanation

This code creates the final alert message sent to Telegram.

It makes the alert readable for a SOC analyst by separating the message into:

```
Detection Summary  
Alert Details  
Network Indicator  
Threat Intelligence  
MITRE ATT&CK Mapping  
Recommended Analyst Action
```

5.5 Telegram HTTP Request Body

Node name: HTTP Request / Send Telegram Alert

Method: POST

URL:

```
https://api.telegram.org/botYOUR_TELEGRAM_BOT_TOKEN/sendMessage
```

Body JSON:

```
{  
  "chat_id": "YOUR_TELEGRAM_CHAT_ID",  
  "text": "={{$json.text}}",  
  "parse_mode": "Markdown"  
}
```

Security Note

The Telegram bot token must never be uploaded to GitHub.

In the public repository, it should be replaced with:

```
YOUR_TELEGRAM_BOT_TOKEN
```

The same rule applies to the AbuseIPDB API key.

6. Wazuh Service Validation Commands

The following commands were used to validate the Wazuh services:

```
sudo systemctl status wazuh-manager --no-pager
sudo systemctl status wazuh-indexer --no-pager
sudo systemctl status wazuh-dashboard --no-pager
```

To check listening ports:

```
sudo ss -tulnp | grep -E ' :443|:5601|:9200'
```

The environment showed Wazuh Dashboard listening on port **443** and Wazuh Indexer listening on port **9200**, which confirmed that the services were running at the system level.

7. Agent Validation Commands

To confirm connected agents:

```
sudo /var/ossec/bin/agent_control -l
```

This command is important because it proves that endpoint devices are connected to the Wazuh Manager.

8. Raw Alert Evidence Commands

Because the dashboard had readiness issues, raw alert validation was used as backup evidence.

Windows Failed Login Evidence

```
sudo grep -i "Logon Failure" /var/ossec/logs/alerts/alerts.js  
on | tail -n 1
```

Kali SSH Failed Login Evidence

```
sudo grep -i "Failed password\|Invalid user\|sshd" /var/osse  
c/logs/alerts/alerts.json | tail -n 3
```

These commands confirm that Wazuh detected real security events even when the dashboard was not fully accessible. The evidence plan also included screenshots for n8n successful execution, Telegram Windows alert, Telegram Kali alert, active agents, and raw `alerts.json` output.

9. Troubleshooting Summary

During the project, the Wazuh Dashboard showed a readiness issue.

The following command was used to test the Wazuh Indexer:

```
curl -k https://127.0.0.1:9200
```

The returned message was:

```
OpenSearch Security not initialized
```

This indicated that the Wazuh Indexer service was running, but the OpenSearch security layer had not been initialized correctly.

Recommended remediation command:

```
sudo /usr/share/wazuh-indexer/bin/indexer-security-init.sh
```

Then restart:

```
sudo systemctl restart wazuh-dashboard
```

If dashboard logs are needed:

```
sudo journalctl -u wazuh-dashboard --no-pager -n 80
```

10. Conclusion

This project successfully demonstrates a complete SOC alert automation workflow.

It combines endpoint monitoring, SIEM detection, automation, threat intelligence enrichment, and real-time notification.

The strongest part of the project is that it is not limited to one endpoint. Any device with a properly installed and enrolled Wazuh Agent can become part of the monitored environment.

The final result is a practical Blue Team automation lab that reflects real SOC concepts such as:

```
Endpoint visibility  
Alert detection  
Log analysis  
Threat intelligence enrichment  
Automated notification  
Incident triage  
Troubleshooting  
Documentation
```

This makes the project suitable for GitHub portfolio documentation, LinkedIn presentation, and cybersecurity learning demonstration.